

## ReadingService.mjs

This is the biggest and most important application file.

Your earlier uploaded version already shows that the service imports database operations such as `findReadingById`, `queryBranchReadings`, `saveReading`, and `updateReading` from the repository.

Its responsibility is:

**Business logic and validation.**

---

## Import `randomUUID`

Your new version starts with:

```
1 import {  
2   randomUUID  
3 } from "node:crypto";
```

This comes from Node.js itself.

It doesn't link to another file in your project.

It gives you:

```
1 randomUUID()
```

which generates your UUIDv4.

For example:

```
1 94458a89-dbd8-43f7-b842-66a63b1d9718
```

---

## Repository imports

Then:

```
1 import {  
2   deleteReadingFromDatabase,  
3   findReadingById,  
4   publishExcursion,  
5   queryBranchReadings,  
6   saveReading,  
7   updateReading  
8 } from "../repositories/readingsRepositories.mjs";
```

This is extremely important.

It means:

```
1 readingsService.mjs
2
3   ── findReadingById()
4   ── saveReading()
5   ── queryBranchReadings()
6   ── updateReading()
7   ── deleteReadingFromDatabase()
8   ── publishExcursion()
9
10      ↓
11 readingsRepositories.mjs
```

The service therefore doesn't create `PutCommand` or `GetCommand` directly.

---

## AppError

```
1 export class AppError
2   extends Error {
```

This creates your own type of error.

A normal JavaScript error might simply say:

```
1 Something broke
```

but your application needs to know:

```
1 Was it 400?
2 Was it 404?
3 Was it 409?
```

So:

```
1 new AppError(
2   404,
3   "Reading not found"
4 )
```

contains:

```
1 statusCode = 404
2 message = Reading not found
```

The service throws it:

```
1 Service
2   ↓
3 throw AppError
4   ↓
5 Controller
6   ↓
7 catch
8   ↓
9 HTTP response
```

---

## validateBodyObject()

This checks that the POST/PATCH body is actually an object.

For example, this is valid:

```
1 {  
2   "maxTempC": 12  
3 }
```

But things such as:

```
1 []
```

or:

```
1 null
```

shouldn't be treated as a valid reading body.

Your original service already performed this object validation before applying the other business rules.

---

## normaliseUuidV4()

This is used for:

```
1 GET one  
2 PATCH  
3 DELETE
```

It does **not generate** a UUID.

That's an important distinction.

It receives an existing URL value:

```
1 94458a89-dbd8-43f7-b842-66a63b1d9718
```

and asks:

Does this look like a UUID version 4?

So:

```
1 POST  
2 → randomUUID()  
3 → CREATE new UUID  
4  
5  
6 GET/PATCH/DELETE  
7 → normaliseUuidV4()  
8 → CHECK existing UUID
```

---

## normaliseDate()

This receives a date:

```
1 2026-08-31T12:00:00.000Z
```

and uses:

```
1 Date.parse()
```

to check it.

If it's valid, it converts it into a standard ISO date:

```
1 new Date(timestamp)
2   .toISOString();
```

This makes all saved dates consistent.

The earlier service used this same normalisation approach before running date-range logic.

---

## getReadingStatus()

This is your core cold-chain business rule:

```
1 if (
2   minTempC < 2 ||
3   maxTempC > 8
4 ) {
5
6   return "excursion";
7 }
8
9 return "ok";
```

So:

```
1 3°C → 7°C
2   ↓
3 ok
4
5
6 1°C → 7°C
7   ↓
8 excursion
9
10
11 3°C → 12°C
12   ↓
13 excursion
```

It also makes sure the values are actual numbers.

This absolutely belongs in the **service** because it is a business rule, not a database rule.

---

## getBranchReadings()

This services:

```
1 GET /branches/{branchId}/readings
```

First it checks:

```
1 Do we have a branchId?
```

Then it handles optional:

```
1 from
2 to
```

For example:

```
1 ?from=2026-08-01&to=2026-08-31
```

Both dates go through:

```
1 normaliseDate()
```

Then it checks:

```
1 from <= to
```

Finally:

```
1 return queryBranchReadings({
2   branchId,
3   from,
4   to
5 });
```

This is where control moves from:

```
1 SERVICE
2   ↓
3 REPOSITORY
```

Your uploaded service shows this exact pattern: validate `branchId` , normalise the optional date range, then call `queryBranchReadings` .

---

## getReadingById()

This handles:

```
1 GET one
```

It gets:

```
1 branchId
2 readingId
```

First:

```
1 normaliseUuidV4(  
2   readingIdValue  
3 );
```

Then:

```
1 await findReadingById(  
2   readingId  
3 );
```

That jumps into:

```
1 readingsRepositories.mjs
```

The repository retrieves the item from DynamoDB.

Then the service checks:

```
1 reading.branchId !== branchId
```

Why?

Imagine this exists:

```
1 readingId = ABC  
2 branchId = BRANCH-002
```

and somebody requests:

```
1 /branches/BRANCH-001/readings/ABC
```

Although UUID `ABC` exists, it doesn't belong to `BRANCH-001`.

So you return:

```
1 404 Reading not found
```

---

## createReading()

This handles POST and is probably the most important function.

The flow is:

```
1 createReading()  
2   ↓  
3 validate body  
4   ↓  
5 check required fields  
6   ↓  
7 validate temperatures  
8   ↓  
9 validate recordedBy  
10  ↓  
11 validate recordedAt  
12  ↓
```

```
13 calculate status
14     ↓
15 generate UUIDv4
16     ↓
17 create item
18     ↓
19 saveReading()
20     ↓
21 possibly publish SNS
```

---

## Required fields

The client now supplies:

```
1 const requiredFields = [
2   "recordedAt",
3   "minTempC",
4   "maxTempC",
5   "recordedBy"
6 ];
```

Notice:

```
1 readingId
```

is NOT there anymore.

That's because Lambda generates it.

---

## Client cannot supply `readingId`

We deliberately check:

```
1 if (
2   body.readingId !== undefined
3 ) {
4
5   throw new AppError(
6     400,
7     "readingId is generated automatically and must not be supplied"
8   );
9 }
```

So the contract is:

```
1 Client controls:
2 recordedAt
3 minTempC
4 maxTempC
5 recordedBy
6
7
8 Server controls:
9 readingId
10 branchId
11 status
12 alertRaisedAt
```

---

## UUID generation

Then:

```
1 const readingId =  
2   randomUUID();
```

This creates UUIDv4.

So:

```
1 Postman  
2 does NOT know ID yet  
3   ↓  
4 POST  
5   ↓  
6 Lambda  
7   ↓  
8 randomUUID()  
9   ↓  
10 ID created  
11  ↓  
12 saved  
13  ↓  
14 returned to Postman
```

---

## Building the DynamoDB item

Then:

```
1 const item = {  
2   readingId,  
3   branchId,  
4   recordedAt,  
5   minTempC: body.minTempC,  
6   maxTempC: body.maxTempC,  
7   recordedBy: body.recordedBy.trim(),  
8   status  
9 };
```

This is the object that gets sent towards DynamoDB.

---

## saveReading()

Then:

```
1 await saveReading(  
2   item  
3 );
```

Remember — `saveReading()` is **not defined in the service**.

It's imported from:

```
1 readingsRepositories.mjs
```



So:

```
1 createReading()  
2   ↓  
3 saveReading()  
4   ↓  
5 Repository  
6   ↓  
7 PutCommand  
8   ↓  
9 DynamoDB
```

---

## 409 protection

The repository uses a DynamoDB condition based on:

```
1 readingId
```

If DynamoDB says the ID already exists, the service converts that AWS error into:

```
1 409 Conflict
```

This is now only a collision safety mechanism because UUIDv4 is server-generated.

---

## Excursion publishing

After saving:

```
1 if (  
2   status === "excursion"  
3 ) {
```

the service calls:

```
1 await publishExcursion({  
2   branchId,  
3   readingId,  
4   recordedAt,  
5   minTempC,  
6   maxTempC,  
7   status  
8 });
```

Again:

```
1 publishExcursion()
```

comes from:

```
1 readingsRepositories.mjs
```

That function talks to SNS.

So:

```
1 Service
```

```
2 "this IS an excursion"
3   ↓
4 Repository
5 "publish this to SNS"
6   ↓
7 SNS
```

This is a good separation:

```
1 SERVICE decides WHETHER
2
3 REPOSITORY handles HOW
```

---

## patchReading()

PATCH is slightly more complex.

Suppose DynamoDB currently contains:

```
1 minTempC = 3
2 maxTempC = 7
3 status = ok
```

Client sends:

```
1 {
2   "maxTempC": 12
3 }
```

First the service validates the UUID.

Then it checks which fields were supplied:

```
1 Object.keys(body)
```

Only these are allowed:

```
1 new Set([
2   "minTempC",
3   "maxTempC",
4   "recordedBy"
5 ]);
```

Therefore this would fail:

```
1 {
2   "readingId": "different"
3 }
```

because the primary identity shouldn't be mutable.

---

## Getting existing values

PATCH needs the existing record first:

```
1 const existing =
```

```
2   await findReadingById(  
3     readingId  
4   );
```

Then:

```
1   const minTempC =  
2     body.minTempC !== undefined  
3     ? body.minTempC  
4     : existing.minTempC;
```

means:

If PATCH supplied a new minimum, use it. Otherwise keep the old minimum.

The same happens for:

```
1   maxTempC  
2   recordedBy
```

So:

```
1   OLD  
2   min = 3  
3   max = 7  
4  
5   PATCH  
6   max = 12  
7  
8   NEW  
9   min = 3  
10  max = 12
```

---

## Recalculate status

The service then calls:

```
1   getReadingStatus(  
2     minTempC,  
3     maxTempC  
4   );
```

That's important.

The client never directly sends:

```
1   {  
2     "status": "excursion"  
3   }
```

The server calculates it.

So:

```
1   max 7  
2   ↓  
3   ok
```

```
4
5 PATCH max 12
6 ↓
7 getReadingStatus()
8 ↓
9 excursion
```

---

## Repository update

Then:

```
1 await updateReading({
2   readingId,
3   minTempC,
4   maxTempC,
5   recordedBy,
6   status
7 });
```

moves into:

```
1 readingsRepositories.mjs
```

which performs the DynamoDB `UpdateCommand` .

---

## PATCH excursion detection

Then you have this kind of logic:

```
1 if (
2   existing.status !== "excursion" &&
3   status === "excursion"
4 ) {
```

This is checking for a **transition**:

```
1 OLD      NEW
2 ok      → excursion
```

If that happens:

```
1 publishExcursion(...)
```

runs.

But:

```
1 excursion → excursion
```

doesn't need another new excursion publication from that PATCH rule.

---

`deleteReading()`

DELETE gets:

```
1 | branchId  
2 | readingId
```

It validates the UUID and then:

```
1 | await findReadingById(  
2 |   readingId  
3 | );
```

If it doesn't exist:

```
1 | 404
```

If it exists but belongs to another branch:

```
1 | 404
```

Otherwise:

```
1 | await deleteReadingFromDatabase(  
2 |   readingId  
3 | );
```

That passes control into the repository.

Repository:

```
1 | DeleteCommand  
2 |   ↓  
3 | DynamoDB
```

Then controller eventually returns:

```
1 | 204 No Content
```